

Feature Prototype

Music Works Catalogue Portal

Computer Science Final Year Project | April–August 2026

Source code and full commit history: https://github.com/AlinaNesterak/FinalProject_UoL.git

1. Project and Template

This project is based on the “**Working with Works — A New Approach to Music Cataloguing**” template. The project builds a modern, catalogue-agnostic web portal for exploring thematic catalogues of classical composers’ works that are currently encoded in MEI (Music Encoding Initiative) XML. These catalogues were previously published by MerMEId, a system built by the Danish Royal Library in 2014, whose publication pipeline has since broken down — leaving valuable scholarly data encoded but inaccessible. The full project transforms this MEI XML into a queryable database, serves it through a documented REST API, presents it through an interactive web interface, and adds a Works Discovery Engine that finds musical connections between works by comparing their incipits (opening notes).

2. How This Prototype Fits Into the Project

The prototype implements and connects the backend half of the full pipeline: parsing, storage, the REST API, and a first version of the discovery engine. It deliberately targets the project’s single greatest technical risk — reliably parsing the complex MEI XML data model — following the principle of building the whole pipeline end-to-end early, even where individual components remain simple, so that feasibility is proven before time is invested in polish. The web frontend is intentionally deferred; the API is exercised directly and through automated tests. Figure 1 shows how the prototype’s components map onto the pipeline.

Prototype pipeline: MEI XML → parser → database → API (with discovery engine)

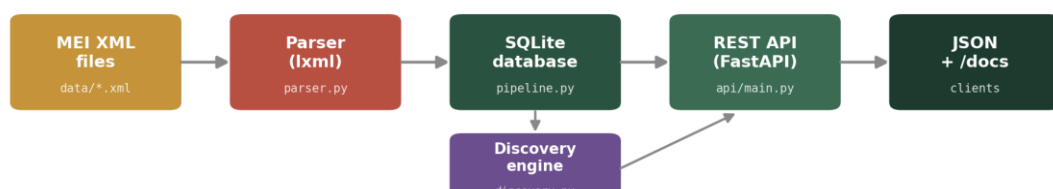


Figure 1: The prototype pipeline. MEI XML is parsed, stored in SQLite, and served by the API; the discovery engine reads incipit data from the database.

3. Features Implemented

- MEI XML parser — extracts multilingual titles, composer, catalogue number, opus, genre, date, instrumentation, and the incipit (tempo, key, meter, and the note-by-note pitch sequence). It is catalogue-agnostic and defensive against missing or malformed data.
- Transformation pipeline — converts a directory of MEI files into a normalised SQLite database, reporting parse errors separately rather than failing.
- REST API — a FastAPI application with endpoints to list, filter, search, and retrieve works, plus a discovery endpoint. Interactive OpenAPI documentation is generated automatically.
- Works Discovery Engine — a first “black box” implementation that scores melodic similarity between works from their incipit pitch sequences.
- Automated test suite — 15 pytest tests covering the parser, pipeline, and every API endpoint, including edge cases.

4. Algorithms, Techniques and Methods

The parser uses the `lxml` library to navigate the MEI XML tree via namespaced XPath queries. MEI is deeply nested and uses optional elements inconsistently, so the parser is written defensively: every field is extracted through a helper that returns an empty value if the element is absent, and malformed files raise a single controlled exception that the pipeline catches. The catalogue is identified from the record’s identifier type (CNW, DCW, and so on), which is what makes the parser catalogue-agnostic — no code changes are needed to ingest a new composer.

The Works Discovery Engine applies a standard music-information-retrieval technique: interval-based melodic comparison. Each incipit’s pitch sequence is converted to a sequence of intervals (the differences between successive pitches), which makes the comparison transposition-invariant — two melodies with the same shape in different keys are treated as identical. Similarity is then computed by combining the edit distance between interval sequences with a coarser melodic-contour comparison (whether each step moves up, down, or stays level). The result is a score between 0 and

1. The internals are deliberately simple so that the black box can be replaced with a richer measure later without changing the interface.

5. Explanation of Key Code

The parser’s defensive design rests on a small helper that safely extracts text from any element, returning `None` when the element is missing. This single function is why incomplete records do not crash the pipeline:

```
def _text(el):
    if el is None:
        return None
    txt = el.text
    return txt.strip() if txt and txt.strip() else None
```

The incipit’s pitch sequence — the data the discovery engine depends on — is built by walking every note element and combining its pitch name, accidental, and octave into a scientific pitch string such as “F#5”:

```
for note in _findall(incip_el, './mei:note'):
    pname = note.get('pname')
    octave = note.get('oct')
    accid = note.get('accid', '')
    if pname and octave:
        symbol = _ACCID.get(accid, '')
        inc.pitches.append(f'{pname.upper()}{symbol}{octave}')
```

In the discovery engine, converting pitches to intervals is what delivers transposition-invariance. The similarity function combines interval and contour edit distances into a single normalised score:

```
def to_intervals(pitches):
    midi = [pitch_to_midi(p) for p in pitches if ...]
```

```

    return [b - a for a, b in zip(midi, midi[1:])]

interval_sim = 1 - edit_distance(ia, ib) / max(len(ia), len(ib))

contour_sim = 1 - edit_distance(ca, cb) / max(len(ca), len(cb))

return (interval_sim + contour_sim) / 2

```

The API layer is thin by design: each endpoint queries SQLite and returns Pydantic models, which FastAPI validates and documents automatically. For example, the discovery endpoint simply delegates to the engine’s `find_similar` function, keeping the web layer and the algorithm cleanly separated.

6. Visual Representation of Results

Running the API and requesting a single work returns clean, structured JSON. The response below for Carl Nielsen’s “Greeting” (CNW 131) shows the multilingual titles, instrumentation, and the extracted incipit pitch sequence — all recovered from the raw MEI XML:

```

{
  "work_id": "CNW 131", "catalogue": "CNW",
  "title_main": "Greeting", "title_da": "Hilsen",
  "composer": "Carl Nielsen", "opus": "10.6",
  "genre": "Song",
  "instrumentation": ["voice", "pianoforte (piano)"],
  "incipit_key": "D major", "incipit_meter": "6/8",
  "incipit_pitches": ["D5", "D5", "D5", "F#5", "A5", "D6"]
}

```

The evaluation script measures how completely each field was recovered across the parsed works (Figure 2).

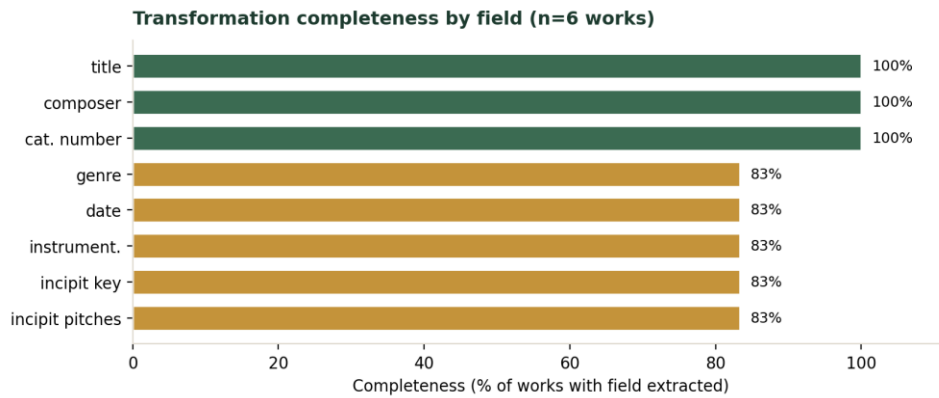


Figure 2: Field-extraction completeness. Core fields reach 100%; optional fields reach 83%, the single gap being a record that legitimately omits them.

7. Evaluation and Intended Improvements

The prototype is evaluated using methods appropriate to a data-transformation and information-retrieval system rather than, for example, machine-learning accuracy metrics. Three measures are used. First, parse robustness: all six valid files parsed successfully and the one deliberately malformed file was rejected cleanly, giving 85.7% robustness with no crash. Second, transformation completeness (Figure 2): 100% on core bibliographic fields and 83% on optional fields, where the gap is a minimal record that genuinely omits them — the parser handles this gracefully rather than inventing data. Third, a discovery sanity check.

The discovery check produced an honest negative result. Two songs by the same poet were expected to score as more similar to each other than to a symphony; in fact the song–symphony pair scored marginally higher (0.300 versus 0.200). With only five to six notes per incipit, the current similarity measure is too coarse to separate works reliably. This is reported deliberately: identifying it now, while the discovery engine is still a deferrable stretch goal, is precisely the value of an early prototype.

Overall the prototype succeeds in its primary purpose — it proves the pipeline is feasible. Real MEI XML is parsed, stored, and served as documented, tested JSON, and the catalogue-agnostic design is validated across two composers (Nielsen and Delius). The intended improvements are clear: run the pipeline against the full ~750-work Nielsen catalogue to obtain statistically meaningful completeness

figures; replace the discovery engine’s similarity measure with a richer approach incorporating rhythm and melodic n-grams over longer excerpts, validated against works with known relationships; and build the React frontend so the data can be evaluated for usability with representative users.

8. Development Plan: Sprints

From this prototype onward, development follows an Agile approach organised into nine two-week sprints across the April–August semester. Each sprint delivers a working, testable increment, so the system is always in a runnable state and progress can be reviewed against concrete deliverables. The core sprints (1–6) build the critical path — pipeline, API, and frontend — while the later sprints (7–8) tackle the ambitious discovery and linked-data features that can be scaled back without endangering a working submission. Figure 3 shows the schedule; the table that follows sets out each sprint’s goal and its definition of done.

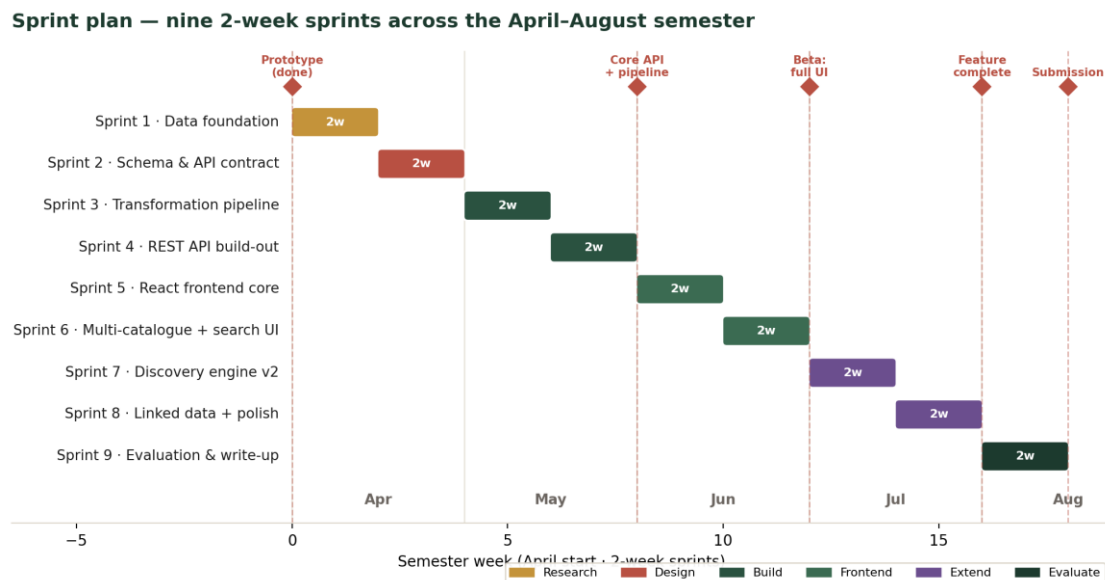


Figure 3: Nine two-week sprints mapped to the April–August semester, with milestones.

Sprint	Goal	Definition of done (deliverable)
1 (Apr)	Data foundation	Full Nielsen + Delius MEI files gathered; parser runs across the whole set; parse-error report generated

Sprint	Goal	Definition of done (deliverable)
2 (Apr–May)	Schema & API contract	Finalised SQLite schema; OpenAPI contract agreed and documented; endpoint stubs returning typed responses
3 (May)	Transformation pipeline	Complete XML→SQLite pipeline over the full catalogue; $\geq 90\%$ field completeness on core fields; unit tests green
4 (May–Jun)	REST API build-out	All endpoints live (list, filter, search, detail, similar); pagination; error handling; API tests green in CI
5 (Jun)	React frontend core	Browse and work-detail views consuming the API; responsive layout; first end-to-end (Playwright) test
6 (Jun–Jul)	Multi-catalogue + search UI	Search and filter UI; catalogue switching; both catalogues browsable through one interface (beta milestone)
7 (Jul)	Discovery engine v2	Richer similarity measure (rhythm + n-grams); sanity check passes on known-related works; results surfaced in UI
8 (Jul–Aug)	Linked data + polish	Wikidata/MusicBrainz identifiers in API and UI; accessibility pass; visual polish (feature-complete milestone)
9 (Aug)	Evaluation & write-up	Usability study run; MEI evaluation report written; final dissertation completed and submitted

Each sprint ends with a short review against its definition of done and a brief retrospective, feeding any incomplete items forward. Because every increment is runnable and tested, the project can be submitted at the end of any sprint with a coherent, working system — the later sprints add ambition, not basic functionality.

9. Testing Strategy

Testing is treated as a first-class, continuous activity rather than a final phase, directly addressing the absence of testing that the literature identifies in the original MerMEId system. The prototype already ships 15 passing tests; the strategy below defines how test coverage grows with each layer, run automatically on every commit through continuous integration.

9.1 Test levels

- Unit tests (pytest) — the parser and discovery engine: correct field extraction, transposition-invariance of similarity, and graceful handling of missing, null, and malformed data. These are the fastest and most numerous tests.
- Integration tests (pytest) — the transformation pipeline end-to-end: a directory of MEI files produces a correct database, with parse errors reported rather than fatal.
- API tests (pytest + FastAPI TestClient) — every endpoint: correct status codes (200 for success, 404 for missing works, 422 for invalid queries), correct JSON shape via Pydantic, pagination, and search recall on known titles.
- End-to-end tests (Playwright) — the React frontend: a user can browse, open a work, search, and switch catalogues; the UI degrades gracefully when a record is incomplete.

9.2 What we check, and how

Property tested	Method	Success criterion
Transformation accuracy	Compare parsed fields against source XML for a sample	$\geq 95\%$ of core fields match the source exactly
Parser robustness	Feed malformed / minimal records	No crash; errors reported and isolated
API correctness	Automated request/response assertions	Correct status codes and JSON schema on all endpoints
Search quality	Query known titles/composers	Expected works appear in results (recall)

Property tested	Method	Success criterion
Discovery validity	Rank known-related vs unrelated works	Related pairs score higher than unrelated
Regression safety	Full suite runs in CI on every commit	No previously-passing test breaks

Coverage is tracked so that the core transformation and API modules stay above a set threshold. Every sprint’s definition of done includes “tests green in CI”, which means a feature is not considered complete until it is covered and the whole suite still passes.

10. Improving the User Experience

Because the project serves both non-technical users (students, enthusiasts) and technical ones (developers, researchers), user experience is a distinct concern from technical correctness. The following improvements are planned as the frontend is built in Sprints 5–6 and refined in Sprint 8.

- No XML required — the interface exposes search, filtering, and browsing so that a user never has to understand MEI or read raw data. This is the core accessibility goal drawn from the project’s user research.
- Progressive disclosure — a work’s summary is shown first; manuscripts, sources, and incipit detail expand on demand, so casual users are not overwhelmed while scholars can still reach depth.
- Honest uncertainty — where a record is incomplete, the UI shows what is missing rather than hiding it, and discovery-engine suggestions are clearly labelled as computational, not scholarly, claims.
- Accessibility — the interface targets WCAG 2.1 AA: keyboard navigation, screen-reader labels, and sufficient colour contrast, audited in Sprint 8.
- Responsiveness and speed — a component-based React layout that works on mobile and desktop, backed by the API’s pagination so large result sets load quickly.

These improvements will be validated in Sprint 9 through a small usability study: representative users complete realistic tasks (for example, “find all works for solo piano composed before 1900”), measured by task-completion rate and think-aloud feedback. Because no user studies of thematic catalogue interfaces exist in the literature, even a small study contributes evidence about what such an interface should offer — feeding directly into the final iteration.